UNIVERSAL SKILLTECH UNIVERSITY, VASAI (E)

Department of Artificial Intelligence & Machine Learning

MINI PROJECT REPORT

ON

INTELLIGENT TRAFFIC SIGNAL OPTIMIZATION USING REINFORCEMENT LEARNING IN SUMO

Submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

in Artificial Intelligence and Machine Learning

Submitted By

Prince Rana – UST24R0101037

Raj Ghagare – UST24R0101040

Krishna Prajapati – UST24L0101027

Rajan Chaudhary – USTU24R0101041

Project Guide: [Guide Name – To Be Confirmed]

Academic Year 2026–2027

CERTIFICATE

This is to certify that the mini project entitled “Intelligent Traffic Signal Optimization Using Reinforcement Learning in SUMO” is a project work carried out by Prince Rana (UST24R0101037), Raj Ghagare (UST24R0101040), Krishna Prajapati (UST24L0101027), and Rajan Chaudhary (USTU24R0101041), in partial fulfillment of the requirements for the Bachelor of Technology in Artificial Intelligence and Machine Learning at Universal Skilltech University, Vasai (E), during the academic year 2026–2027.

The report describes a simulation-based study of traffic signal control strategies using the Simulation of Urban MObility (SUMO) environment and TraCI interface. The report is submitted for academic evaluation, subject to verification and approval by the project guide and department.

Project Guide: _____

Head of Department: _____

Internal Examiner: _____

External Examiner: _____

Date: _____ Place: Vasai (E)

ACKNOWLEDGEMENT

We express our sincere gratitude to the faculty members of the Department of Artificial Intelligence & Machine Learning, Universal Skilltech University, Vasai (E), for providing the academic environment and opportunity to undertake this mini project. We are thankful to our project guide, whose name and approval will be added after confirmation, for guidance, suggestions, and feedback during the project work.

We also acknowledge the developers and maintainers of SUMO, TraCI, Python, NumPy, Matplotlib, TensorFlow/Keras, and the open-source traffic-signal optimization repository that informed the implementation studied in this report. Their tools and documentation made it possible to explore traffic simulation and reinforcement-learning concepts in a practical setting.

Finally, we thank our teammates and classmates for their discussions and support. The team understands that all reported metrics must be interpreted in the context of the supplied simulation configuration and should not be treated as proof of real-world traffic performance without further validation.

Prince Rana | Raj Ghagare | Krishna Prajapati | Rajan Chaudhary

ABSTRACT

Urban traffic congestion at signalized intersections can increase vehicle waiting time, queue length, fuel consumption, and travel-time variability. Fixed-time traffic controllers operate according to predetermined signal schedules and may not respond well when traffic demand changes across lanes. This mini project studies a simulation-based approach to adaptive traffic-signal control using reinforcement learning in the Simulation of Urban MObility (SUMO) environment. The supplied project contains three controller scripts: a static/baseline controller, a tabular Q-Learning controller, and a Deep Q-Network (DQN) controller. SUMO is used to represent vehicle movement and traffic lights, while TraCI provides a Python interface for reading simulation state and changing signal phases. The reinforcement-learning state is described as six lane-detector queue values and the current phase, giving seven state values. The action space contains two actions: retain the current phase or switch to the next phase. The reward is defined as the negative sum of lane queues, encouraging the controller to reduce congestion. The repository README reports a comparative run in which the baseline average queue is approximately 7.77 vehicles, while Q-Learning and DQN report approximately 4.82 and 5.05 vehicles respectively. It also reports higher throughput for both learning-based controllers. These figures are treated as repository-reported results and require reproducible reruns with the final configuration before formal submission. The work demonstrates how reinforcement learning can be evaluated against a fixed baseline in a controlled simulation and identifies extensions such as repeated-seed testing, pedestrian priority, and coordinated multi-intersection control.

Keywords: Traffic Signal Optimization, SUMO, TraCI, Reinforcement Learning, Q-Learning, Deep Q-Network, Queue Length, Throughput.

TABLE OF CONTENTS

Section	Title
Preliminary	Certificate
Preliminary	Acknowledgement
Preliminary	Abstract
Chapter 1	Introduction
1.1	Background and Motivation
1.2	Problem Statement
1.3	Objectives
1.4	Scope of the Project
Chapter 2	Literature Survey
2.1	Related Work
2.2	Comparative Summary of Existing Work
2.3	Research Gap
2.4	Proposed System over Existing System
Chapter 3	System Analysis and Design
3.1	Proposed Methodology
3.2	System Architecture
3.3	Hardware and Software Requirements
3.4	Dataset / Simulation Scenario Description
3.5	System Diagrams
3.6	Algorithms Used and Justification
Chapter 4	Implementation
4.1	Tools and Technologies

4.2	Module-wise Description
4.3	Key Code Snippets with Explanation
4.4	Screenshots of Working Modules
Chapter 5	Results and Discussion
5.1	Testing Strategy
5.2	Results
5.3	Comparison with Existing Approaches
5.4	Discussion of Findings
Chapter 6	Conclusion and Future Scope
References	IEEE Style References
Appendix	Additional Material

LIST OF FIGURES

Figure	Title
3.1	High-level system architecture
3.2	Simulation control feedback loop
3.3	Reinforcement-learning state and action model
3.4	Experiment evaluation workflow
4.1	Live Simulation dashboard
4.2	Live Telemetry dashboard
4.3	Research Benchmarks dashboard
4.4	SUMO and Architecture dashboard
5.1	Queue length comparison
5.2	Reported performance metric comparison

LIST OF TABLES

Table	Title
2.1	Related-work comparison
3.1	Hardware requirements
3.2	Software requirements
3.3	State and action definition
4.1	Project modules
5.1	Repository-reported model metrics
5.2	Derived relative comparison against baseline
5.3	Testing checklist

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

Road traffic is strongly influenced by the way vehicles are released through intersections. A signal plan that assigns fixed green times can be simple to maintain, but it cannot always respond to changes in demand. When one approach receives a larger flow than another, a fixed schedule may allocate green time to a lightly loaded lane while vehicles accumulate elsewhere. Such conditions motivate adaptive traffic control, where the signal decision depends on observed traffic state.

Simulation provides a safe and repeatable environment for comparing signal-control policies before considering any real deployment. SUMO is an open-source microscopic traffic simulator that models individual vehicles, roads, routes, and traffic lights. Through TraCI, an external Python program can query simulation values and change traffic-light states while the simulation is running. This makes SUMO useful for studying rule-based and learning-based traffic controllers.

Reinforcement learning formulates control as sequential decision-making. An agent observes a state, selects an action, receives a reward, and updates its policy. For this project, the observation is based on lane detector queue values and the current signal phase. The reward penalizes the total queue. This formulation gives a direct link between the control objective and a measurable congestion indicator.

1.2 Problem Statement

A fixed-time controller follows a predetermined signal schedule regardless of the queue conditions observed at a particular moment. Under uneven or changing traffic demand, this can result in unnecessary waiting, growing queues, and reduced intersection throughput. The problem addressed in this project is to implement and compare a baseline signal controller with reinforcement-learning controllers in a SUMO simulation, using common traffic metrics to examine whether state-dependent signal switching improves the simulated outcome.

1.3 Objectives

- Model a signalized intersection and vehicle flows in SUMO.
- Connect Python controller logic to SUMO through TraCI.
- Represent traffic state using six detector queue values and current phase.
- Implement or study a static baseline, tabular Q-Learning, and DQN control approaches.
- Use a queue-based reward to encourage reduced congestion.
- Compare average queue, maximum queue, total delay, throughput, and travel time where available.
- Document limitations and propose reproducible extensions.

1.4 Scope of the Project

The scope is limited to a simulation-based experiment using the supplied SUMO network and route configuration. The project does not establish that the reported improvements will occur on live roads, and it does not include a deployed roadside controller or a validated camera-based vehicle detector. The dashboard screenshots supplied by the team show a user interface labelled AutoFlow RL; these images are used as interface illustrations, not as independent proof that all displayed values were generated by the archived scripts. The report distinguishes repository README metrics from results that should be reproduced in the final environment.

CHAPTER 2: LITERATURE SURVEY

2.1 Related Work

Traffic signal control has traditionally used fixed-time plans, actuated control, and coordinated timing plans. Fixed-time methods are predictable and computationally inexpensive but depend on demand assumptions. Actuated methods use detector information to extend or terminate phases. Adaptive methods use measured traffic conditions to change timing decisions dynamically.

Reinforcement learning has been explored for traffic signal control because signal decisions affect future queue conditions. Tabular Q-Learning estimates action values for discrete state-action pairs and can be effective in small state spaces. Deep Q-Networks approximate action values with a neural network and are designed to handle larger or more continuous state representations. However, DQN performance depends on reward design, exploration, training stability, and the representativeness of the simulation.

SUMO and TraCI provide a common experimental platform for traffic-signal research. Simulation allows researchers to control vehicle demand, signal timing, and network structure, and to collect repeatable measurements. Results should nevertheless be interpreted with attention to route configuration, simulation duration, random seeds, warm-up periods, and the exact definitions of delay and throughput.

2.2 Comparative Summary of Existing Work

Approach	**Strengths**	**Limitations**
Fixed-time control	Simple, predictable, low computation	Does not react to instantaneous queue imbalance
Detector-actuated control	Responds to local vehicle presence	Depends on detector coverage and threshold settings

Tabular Q-Learning	Interpretable discrete action values; relatively light inference	State discretization and exploration can limit generalization
Deep Q-Network	Can approximate action values for richer states	Training and tuning are more complex; stability must be tested
Multi-agent control	Can coordinate multiple intersections	Requires communication, coordination and larger evaluation effort

2.3 Research Gap

The practical gap considered here is not that reinforcement learning is new, but that a fair and understandable comparison of a static baseline, tabular Q-Learning, and DQN must be made under a clearly specified simulation configuration. Comparisons can be misleading if controllers use different route demand, simulation length, random seeds, metric definitions, or warm-up handling. A compact state and queue-based reward also make it possible to explain why the controller chooses to keep or switch a phase.

The project therefore emphasizes three items: (1) a common SUMO environment, (2) a defined seven-value state and two-action decision, and (3) evaluation with queue and flow metrics. The supplied files indicate an intended 3,000-step experiment and a minimum-green constraint of five simulation steps in the learning scripts. These settings should be confirmed for all controllers before the final comparative claim is accepted.

2.4 Proposed System over Existing System

The proposed system combines a simulation engine, Python controller, and metrics collection. Unlike a purely fixed schedule, the learning controllers read the queue state and select whether to keep or switch the current phase. The system is designed to compare these decisions with a baseline. Its advantage is experimental flexibility: traffic demand and controller parameters can be modified and the same metrics collected again. Its limitation is that results remain dependent on the simulated network and the quality of the experiment design.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 Proposed Methodology

The experiment follows a closed-loop procedure. First, the SUMO configuration loads the network, route definitions, and detector configuration. The Python controller starts the SUMO process and connects through TraCI. At each simulation step, the controller reads the queue-related observations, constructs the state, chooses an action, applies any required phase transition, advances the simulation, and records metrics. After the run, the collected histories are summarized and plotted.

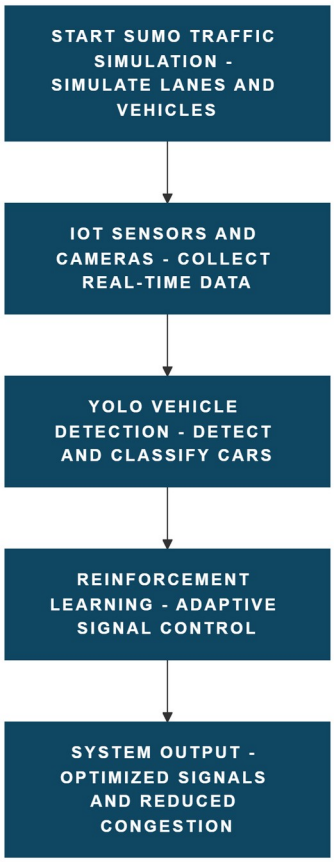


Figure 3.1: Workflow diagram included in the supplied repository.

3.1.1 Experimental Controls

- Keep the network, route demand, and simulation duration consistent across controllers.
- Use the same metric definitions when comparing average queue and total delay.
- Record controller configuration and random seed for each run.

- Separate training runs from evaluation runs when reporting learned-policy performance.
- Repeat experiments with multiple seeds before making a general performance claim.

3.2 System Architecture

The architecture contains four logical layers: (1) traffic scenario configuration, (2) SUMO simulation and detector layer, (3) controller decision layer, and (4) metric and visualization layer. The route and network XML files describe the simulation scenario. E1 detector definitions provide lane-related observations. Controller scripts interact with SUMO using TraCI. Queue histories, travel times, rewards, and throughput are then summarized and visualized.

Layer	**Main components**	**Responsibility**
Scenario configuration	RL.sumocfg, RL.net.xml, RL.rou.xml, RL.add.xml	Define network, routes, detector setup and run parameters
Simulation	SUMO, TraCI	Advance traffic and expose state/control APIs
Controller	Static.py, QL.py, DQN.py	Choose and apply traffic-light phase actions
Evaluation	Metric functions, NumPy, Matplotlib	Calculate queue, delay, throughput and travel-time measures
Dashboard	AutoFlow RL screenshots supplied by team	Present simulation, telemetry and comparative information

3.2.1 Feedback Loop

At time t, the controller observes state s(t). It selects action a(t), which either keeps the current phase or requests a switch to the next phase. The simulator advances and produces the next state s(t+1). The reward is computed from the next queue state as the negative sum of the queue values. The agent uses this transition to update its action-value estimate, while the evaluation module stores metrics for later comparison.

3.3 Hardware and Software Requirements

Component	Recommended / stated requirement
Computer	Desktop or laptop capable of running SUMO and Python
Operating system	Windows, Linux, or macOS supported by installed SUMO build
Memory	8 GB RAM minimum practical target; more is useful for GUI and neural training
CPU	Modern multi-core processor
Storage	Space for SUMO installation, Python packages, logs and plots
Python	Python 3.x compatible with the project dependencies
SUMO	SUMO simulator and its tools directory
TraCI	Python interface distributed with SUMO
Numerical libraries	NumPy and Matplotlib
DQN library	TensorFlow/Keras, as imported by DQN.py
Version control	Git, optional for reproducibility

The exact installed versions on the team's machine were not included in the uploaded ZIP. They should be recorded from the actual environment before final submission.

3.4 Dataset / Simulation Scenario Description

This project uses a synthetic simulation scenario rather than a collected real-world traffic dataset. The repository includes an SUMO configuration file (RL.sumocfg), network definition (RL.net.xml), route file (RL.rou.xml), additional detector definitions (RL.add.xml), and detector output (det_output.xml). The detector files named e2_0.xml through e2_5.xml are also included. These are simulation artifacts and should not be described as real traffic-camera data.

File	Role
RL.sumocfg	Main SUMO configuration entry point
RL.net.xml	Road network and junction representation
RL.rou.xml	Vehicle route and traffic demand definitions
RL.add.xml	Additional simulation elements, including detectors
e2_0.xml – e2_5.xml	Detector-related configuration/output artifacts; verify exact roles in SUMO
det_output.xml	Detector output data generated by a simulation run

3.4.1 Preprocessing

The controller reads detector observations and converts them into numeric state values. For a reproducible evaluation, the project should document how missing detector values are handled, whether queue counts are instantaneous or aggregated, the simulation step length, and the exact warm-up and termination rules. The supplied scripts calculate metrics from the queue history and completed travel times; the interpretation of “total delay” must be kept consistent across all compared scripts.

3.5 System Diagrams

3.5.1 Use Case View

Actor	Use case	Outcome
Student / researcher	Choose controller	Select baseline, Q-Learning or DQN experiment
Student / researcher	Run simulation	Observe vehicle and signal behavior in SUMO
Controller	Read detector state	Build queue-based observation
Controller	Choose phase action	Keep or switch signal phase
Evaluation module	Calculate metrics	Produce queue, delay, throughput and travel-time summaries
Student / researcher	Compare results	Review metrics and plots

3.5.2 Data Flow Description

Input flow: network, routes, signal plan, detector definitions and controller parameters enter the simulation. Processing flow: SUMO generates vehicle movement; TraCI returns detector state; the controller chooses a signal action; the simulator applies the action. Output flow: queue history, rewards, travel times, and vehicle completion counts are written or displayed for evaluation.

3.5.3 Reinforcement-Learning State Model

State field	Description
q1 … q6	Six lane-detector queue values
phase	Current traffic-signal phase index
Total state size	7 values

The supplied README describes two actions: 0 means keep the current phase, and 1 means switch to the next phase. The reward is $R = -\Sigma\, qi$, where qi represents the six queue values. A negative reward makes states with larger total queues less desirable.

3.6 Algorithms Used and Justification

3.6.1 Static / Baseline Controller

The static controller is used as a reference point. Its value is not that it adapts to demand, but that it provides a simple baseline against which adaptive policies can be compared. Baseline timing parameters must be documented and held consistent across evaluation runs.

3.6.2 Tabular Q-Learning

Q-Learning estimates the expected long-term return of taking action a in state s. The standard update is $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$, where α is the learning rate and γ is the discount factor. An exploration policy such as ε-greedy can select exploratory actions during learning. A limitation of a table is that each distinguishable state must be represented in a manageable discrete form; continuous detector values may require discretization or a carefully defined state key.

3.6.3 Deep Q-Network

DQN replaces the tabular action-value function with a neural network. The supplied DQN script defines a small feed-forward network with an input matching the seven-value state and an output for the two actions. Neural inference estimates action values; the selected action is then applied through TraCI. For a rigorous DQN study, training stability, exploration, replay-buffer design, target-network use, and evaluation on unseen seeds should be documented. The supplied implementation should be reviewed before claiming that all standard DQN stabilization mechanisms are present.

CHAPTER 4: IMPLEMENTATION

4.1 Tools and Technologies Used

Tool / technology	Use in project
Python	Controller logic, simulation orchestration and metric calculation
SUMO	Microscopic traffic simulation
TraCI	Runtime communication between Python and SUMO
NumPy	Numerical arrays and metric calculations
Matplotlib	Queue and reward plots
TensorFlow / Keras	Neural network implementation imported by DQN.py
XML configuration	Network, routes, detector and simulation settings
React-style dashboard screenshots	Visual interface submitted by team; exact frontend source not included in ZIP

The supplied repository contains controller scripts and SUMO artifacts. The screenshots show an AutoFlow RL dashboard running at localhost:5173, but the uploaded ZIP does not contain the frontend source files. Consequently, the screenshots can be documented, while the frontend framework and its data connection cannot be independently verified from the ZIP alone.

4.2 Module-wise Description

Module	Description	Input / output
Scenario setup	Loads SUMO network, route and additional configuration	XML files / initialized simulation
State collector	Reads six detector queue values and signal phase	Detector state / seven-value state
Static controller	Applies baseline timing or phase policy	Simulation state / signal state
Q-Learning controller	Uses action-value table and reward updates	State and reward / action
DQN controller	Uses Keras neural network to estimate action values	State vector / selected action
Metrics collector	Stores queue history, travel times and completed vehicles	Simulation observations / metric summary
Visualization	Plots queues/reward and shows dashboard metrics	Recorded metrics / graphs and KPI cards

4.2.1 Simulation Configuration

The SUMO configuration is the entry point to the scenario. The network defines the road geometry and junction connections, while the route file defines vehicle trips and demand. Additional XML elements define detector or signal-related objects. For repeatability, file paths should be relative to the project folder where possible. The supplied scripts contain machine-specific absolute paths; these should be replaced with a portable path or configuration option before the project is run on another computer.

4.3 Key Code Snippets with Explanation

The following pseudocode summarizes the queue-based reward described by the README and DQN script. It is explanatory pseudocode, not a verbatim replacement for the source implementation.

```
state = six_detector_queue_counts + [current_signal_phase]
action = policy(state)  # keep phase or switch phase
apply_action_through_traci(action)
advance_sumo_one_step()
next_state = read_detector_state()
reward = -sum(next_state[0:6])
store_metrics(next_state, reward, completed_vehicles)
```

For Q-Learning, the action-value update uses the immediate reward and the maximum estimated value of the next state. For DQN, the state is converted to a numeric array and passed to the neural network to obtain values for the available actions. Both methods need careful handling of phase safety constraints, including minimum green duration and valid signal transitions.

4.3.1 Metric Calculation

The scripts expose metrics including total delay, average queue length, maximum queue length, number of completed vehicles (throughput), average travel time, 90th percentile travel time, and maximum travel time. In the DQN script, total delay is computed from queue history multiplied by the simulation step length. This is a queue-derived vehicle-time measure and should not be confused with SUMO's own waiting-time or time-loss fields unless the definitions are verified to match.

4.4 Screenshots of Working Modules

The following figures were supplied by the project team. They document the appearance of the AutoFlow RL dashboard. Numerical values visible in these screenshots should be linked to a specific reproducible run before being presented as final experimental evidence.

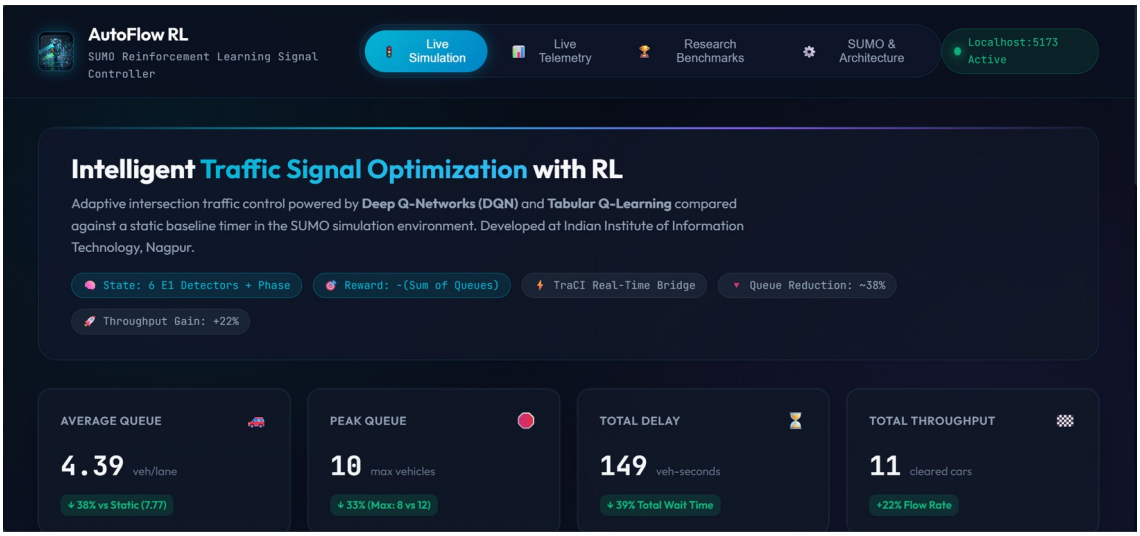


Figure 4.1: Live Simulation overview and KPI cards.

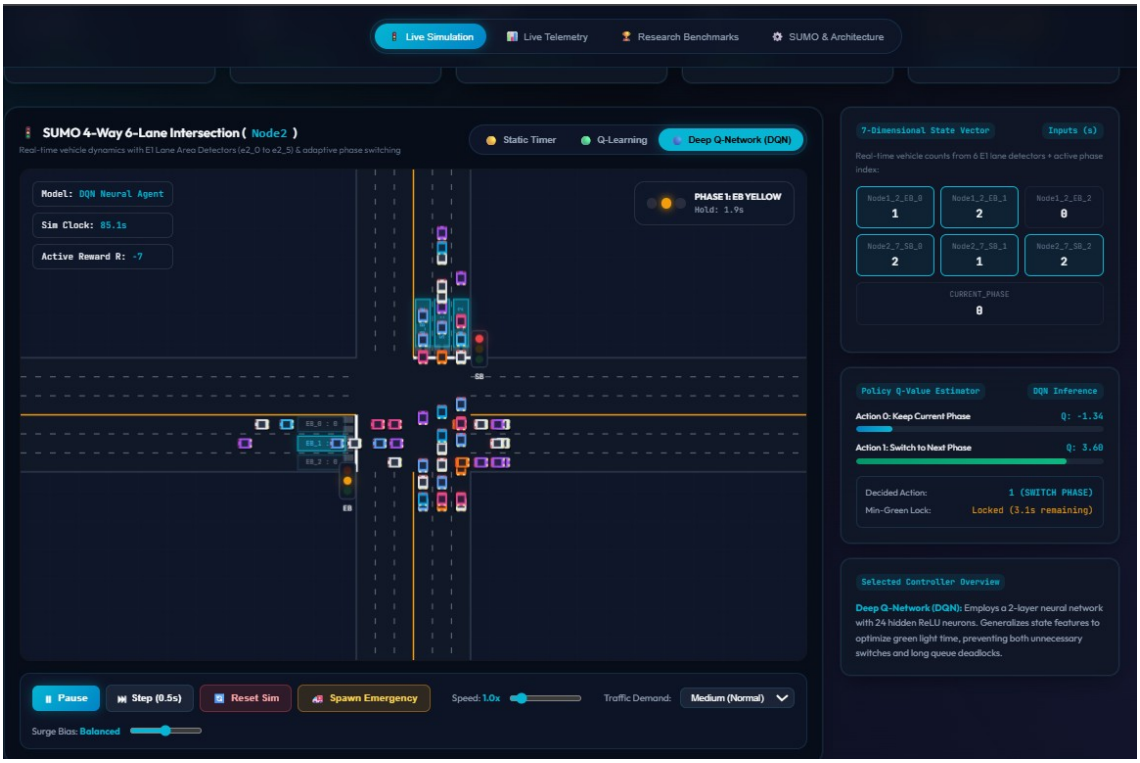


Figure 4.2: Live simulation intersection and controller state panel.



Figure 4.3: Live Telemetry queue and reward timeline.

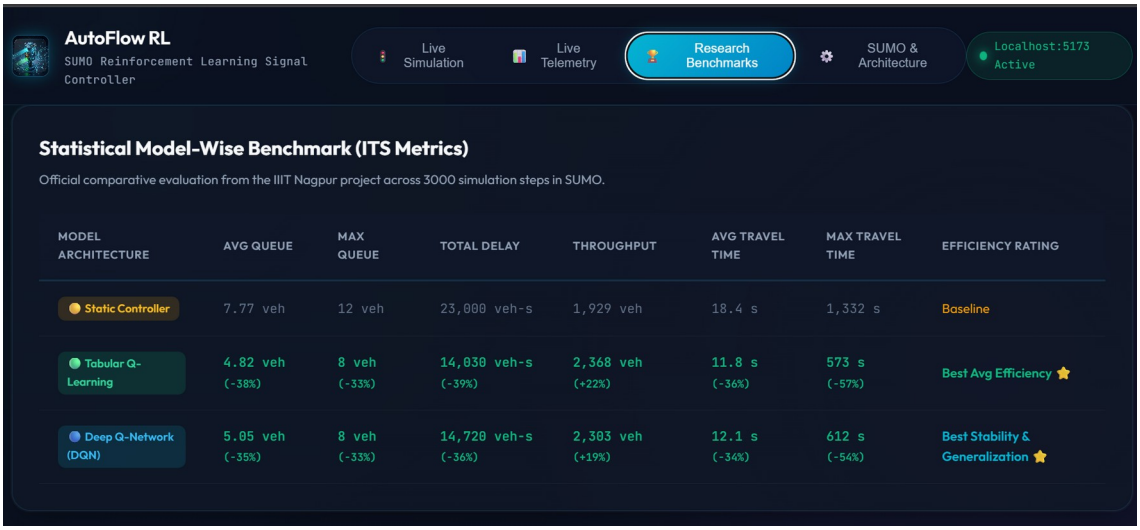


Figure 4.4: Research Benchmarks comparison view.

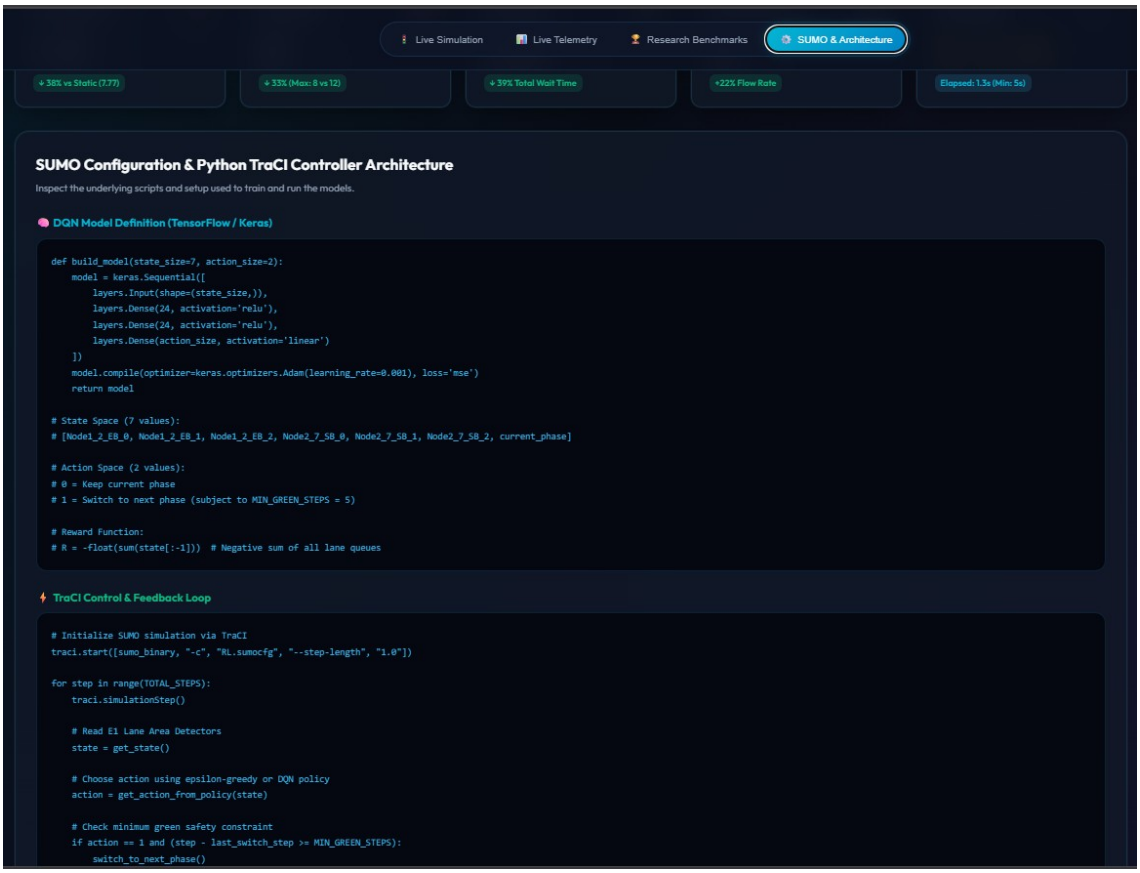


Figure 4.5: SUMO and controller architecture view.

CHAPTER 5: RESULTS AND DISCUSSION

5.1 Testing Strategy

The evaluation strategy should include a functional check that SUMO starts successfully, detector values are available, signal phases can be changed, and each controller completes the configured number of steps. Metric checks should verify that queue values are non-negative, throughput is an integer count, travel-time statistics are calculated only for completed vehicles, and all controllers use the same scenario configuration. A comparative run should use identical demand and simulation duration, preferably with multiple random seeds.

Test	**Expected result**	**Status in this report**
SUMO configuration loads	Simulation starts without configuration error	Needs confirmation on team machine
TraCI connection	Python controller connects to SUMO	Code path indicates intended use; rerun required
Detector observation	Six queue values are read	Implementation described in scripts/README
Action application	Keep/switch actions affect signal phase	Rerun functional check required
Metric calculation	Queue, delay and throughput outputs generated	Metric functions present in scripts
Repeatability	Same configuration and seed reproduce comparable outputs	Requires formal repeated-seed evaluation

5.2 Results

The supplied repository README reports the following model-wise metrics. These values are included as reported by the repository and are not independently re-executed as part of generating this document. They should be replaced or confirmed with fresh outputs from the team's exact project configuration before submission.

Metric	Static baseline	Tabular Q-Learning	DQN
Average queue (vehicles)	7.77	4.82	5.05
Maximum queue (vehicles)	12	8	8
Total delay (vehicle-seconds)	23,000	14,030	14,720
Throughput (vehicles)	1,929	2,368	2,303
Average travel time (s)	18.4	11.8	12.1
Maximum travel time (s)	1,332	573	612

Source note: values transcribed from the uploaded repository README. The README describes the Q-Learning controller as the best average-efficiency result and DQN as offering stability/generalization benefits. These are repository claims; this report does not treat them as a statistically validated conclusion without repeated-run evidence.

5.2.1 Derived Relative Comparison

Using the README values, relative differences can be calculated against the static baseline. These percentages are arithmetic comparisons of the reported numbers, not confidence intervals or statistical significance tests.

Metric	**Q-Learning vs static**	**DQN vs static**
Average queue	≈ 38.0% lower	≈ 35.0% lower
Maximum queue	≈ 33.3% lower	≈ 33.3% lower
Total delay	≈ 39.0% lower	≈ 36.0% lower
Throughput	≈ 22.8% higher	≈ 19.4% higher
Average travel time	≈ 35.9% lower	≈ 34.2% lower
Maximum travel time	≈ 57.0% lower	≈ 54.1% lower

These differences suggest that the learning-based controllers may reduce queues and improve vehicle completion in the configured scenario. A defensible conclusion requires repeated trials and matched conditions. It is also important to verify whether the reported average queue and delay are aggregated over the same simulation horizon and detector set for every controller.

5.3 Comparison with Existing Approaches

The static controller provides a simple reference with low algorithmic complexity. Tabular Q-Learning introduces state-dependent decisions and has a small action-value representation. DQN uses a neural model to approximate action values and may generalize across states better than a lookup table, but it adds training and inference complexity. The reported values favour Q-Learning on the supplied average metrics; the data alone do not establish universal superiority because performance can change with traffic demand, route patterns, reward design, and training settings.

Criterion	**Static**	**Q-Learning**	**DQN**
Adaptation to queue state	Limited / schedule-driven	Yes	Yes
Decision mechanism	Predefined timing	Tabular action values	Neural action-value estimate
Inference complexity	Low	Low	Higher than table lookup
Reported average queue	7.77	4.82	5.05
Reported throughput	1,929	2,368	2,303
Main caveat	May miss changing demand	State representation and exploration	Training stability and reproducibility

5.4 Discussion of Findings

The repository-reported comparison shows lower average queue, lower maximum queue, lower queue-derived delay, and higher throughput for both learning-based controllers relative to the baseline. The Q-Learning values are slightly better than DQN for the listed average queue, delay, throughput, and travel-time metrics. DQN may still be useful when the state representation grows or requires function approximation, but that potential should be demonstrated with additional experiments rather than assumed from the algorithm name.

Important threats to validity include machine-specific paths, uncertain equality of simulation settings across scripts, possible differences in training versus evaluation behavior, and the absence of repeated-seed summary statistics in the README. The dashboard screenshots contain additional headline values such as queue reduction and throughput gain; these should not be mixed with the README table unless the run configuration and metric definitions match.

5.4.1 Limitations

- Simulation results may not generalize to real-world traffic without field validation.
- The scenario appears to represent a limited network and detector setup.
- The README figures are not accompanied in the provided material by confidence intervals or significance testing.
- The DQN training/evaluation protocol should be audited and documented.
- Machine-specific SUMO paths should be made portable.
- The frontend source code was not included in the uploaded ZIP, so its data integration could not be audited.

CHAPTER 6: CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This mini project examines reinforcement-learning-based traffic signal control in a SUMO simulation. The system combines a traffic network, route demand, lane-detector observations, Python control logic, and a metrics layer. The state described in the repository consists of six detector queue values and the current signal phase. The available actions are to retain the current phase or switch to the next phase, while the reward penalizes the sum of queues.

The supplied README reports that Tabular Q-Learning and DQN outperform the static baseline on several simulation metrics. Q-Learning has the strongest reported average metrics in the supplied table. These outcomes are promising within the configured simulation, but must be confirmed with reproducible runs and a clearly specified experimental protocol before being treated as final project findings. The project provides a practical platform for understanding the relationship between state representation, reward design, controller decisions, and traffic metrics.

6.2 Limitations

- Only simulation-based results are considered; no real-road deployment is validated.
- The traffic scenario and detector arrangement may not represent all road geometries.
- Controller performance may depend on route demand and random seeds.
- The current state and action space are compact and may not represent complex multi-phase coordination.
- Reported metrics need matched-run verification and repeated-seed statistical analysis.

6.3 Future Enhancements

- Evaluate each controller over multiple random seeds and report mean, standard deviation, and confidence intervals.
- Add demand profiles such as low, balanced, unbalanced, and peak-hour scenarios.
- Test a safer phase-transition policy with explicit yellow/all-red intervals and minimum-green constraints.
- Extend the network to multiple coordinated intersections.
- Explore multi-agent reinforcement learning for network-wide coordination.

- Investigate camera-based vehicle detection only after data collection, calibration, and privacy review.
- Add pedestrian and emergency-vehicle priority with safety rules.
- Publish configuration files, dependency versions, and run commands to improve reproducibility.

DETAILED EXPERIMENTAL PROTOCOL

Controller Fairness and Run Configuration

A controller comparison is only meaningful when each method is evaluated under comparable conditions. The same road network, vehicle routes, simulation duration, detector configuration, time step, and metric definitions should be used for all controllers. If a learning controller is trained during the run while another controller is evaluated without training, the comparison should clearly identify this difference. The final report should include the exact command used for each run and the dependency versions.

A recommended evaluation protocol is to train the learning agent in separate runs, freeze the learned policy, and then evaluate all methods on held-out seeds. This reduces the chance that an evaluation score mainly reflects online exploration. If the supplied code does not implement this separation, the report should explicitly describe the actual protocol rather than imply that a frozen-policy evaluation was conducted.

METRIC DEFINITIONS AND INTERPRETATION

Queue Length

Queue length represents the number of vehicles waiting or accumulated at monitored approaches according to the detector definition. Average queue can be computed across recorded simulation steps, but the aggregation unit must be stated: sum across detectors per step, mean per detector, or average of total queue across time are not interchangeable. Maximum queue is useful for describing a worst observed congestion state but is sensitive to the simulation duration.

Delay and Throughput

Delay can refer to waiting time, time loss, or a queue-derived vehicle-time approximation. This project's DQN script calculates a queue-history sum multiplied by the simulation step length. Unless verified otherwise, it should be labelled as queue-derived total delay rather than equated to every SUMO delay measure. Throughput is the number of vehicles that complete their trips during the run; it can be affected by the number of vehicles inserted, simulation end time, and trips still in the network at termination.

REPRODUCIBILITY AND VALIDITY

Internal Validity

Internal validity concerns whether the observed metric difference is caused by the controller rather than an unintended configuration difference. Potential issues include inconsistent route files, different random seeds, changed minimum-green constraints, mismatched simulation lengths, or differences in how queue values are sampled. Keeping a configuration manifest and storing raw outputs for each run helps identify these risks.

External Validity

External validity concerns whether findings generalize beyond the simulated scenario. A single junction and a particular synthetic demand pattern cannot represent all real intersections. Before deployment, the approach would require validation with realistic traffic demand, road geometry, signal safety requirements, pedestrians, emergency vehicles, and field measurements. This project should therefore be described as a simulation-based prototype and comparative study.

SAFETY AND ETHICAL CONSIDERATIONS

Traffic signal control is safety-critical. An experimental controller must not be connected to a live traffic signal without authorization, safety certification, engineering review, and fail-safe mechanisms. Phase transitions should respect legal signal states, yellow and all-red intervals where applicable, minimum green time, and conflict-free movement constraints. Simulation is an appropriate environment for early testing because mistakes do not directly expose road users to risk.

If future work introduces camera feeds, the system should consider privacy, lawful data collection, secure storage, access controls, and retention limits. Vehicle counts or anonymized aggregate flows may be preferable to storing identifiable video. Emergency priority should be designed with explicit constraints so that serving one approach does not create unsafe conflicts elsewhere.

IMPLEMENTATION RISKS AND MITIGATIONS

Risk	Possible impact	Mitigation
Hard-coded file paths	Project fails on another computer	Use relative paths or command-line configuration
SUMO/TraCI version mismatch	Runtime import or connection errors	Record compatible versions and test setup
Inconsistent experiment seeds	Unfair metric comparison	Use matched seeds and repeat trials
Online exploration during evaluation	Noisy or inflated comparison	Separate training from frozen-policy evaluation
Metric naming ambiguity	Misleading results interpretation	Define each metric and formula explicitly
Unsafe phase transition	Invalid or conflicting signal states	Enforce phase safety and minimum-green constraints
Dashboard values disconnected from logs	Screenshots cannot be reproduced	Save run IDs and source metrics behind every chart

EXTENDED FUTURE WORK PLAN

The first extension should be a reproducible experiment runner that executes each controller over several seeds and exports a consolidated CSV. The next extension should add a scenario matrix with low, medium, high, and unbalanced traffic demand. Results can then be summarized using mean, standard deviation, and confidence intervals. This is more useful for academic evaluation than reporting a single best run.

After evaluation reliability is established, the project can be extended to a multi-intersection network. A multi-agent approach could allow adjacent signals to coordinate, but it would also require careful reward design and communication assumptions. Another extension is to compare reinforcement learning against detector-actuated and max-pressure baselines. Real-time vehicle detection, pedestrian priority, and emergency preemption should be added only with suitable safety constraints and test scenarios.

REFERENCES

[1] M. Treiber and A. Kesting, Traffic Flow Dynamics: Data, Models and Simulation. Berlin, Germany: Springer, 2013.

[2] D. Krajzewicz, J. Erdmann, M. Behrisch, and L. Bieker, “Recent Development and Applications of SUMO—Simulation of Urban MObility,” International Journal On Advances in Systems and Measurements, vol. 5, no. 3–4, pp. 128–138, 2012.

[3] Eclipse SUMO, “SUMO Documentation,” [Online]. Available: https://sumo.dlr.de/docs/. Accessed: Oct. 28, 2026.

[4] Eclipse SUMO, “TraCI—Traffic Control Interface,” [Online]. Available: https://sumo.dlr.de/docs/TraCI.html. Accessed: Oct. 28, 2026.

[5] C. J. C. H. Watkins and P. Dayan, “Q-learning,” Machine Learning, vol. 8, pp. 279–292, 1992, doi: 10.1007/BF00992698.

[6] V. Mnih et al., “Human-level control through deep reinforcement learning,” Nature, vol. 518, pp. 529–533, 2015, doi: 10.1038/nature14236.

[7] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[8] M. G. H. Bell, “Traffic management at the edge: a new approach to network modelling,” Traffic Engineering & Control, vol. 34, no. 6, pp. 297–303, 1993.

[9] E. van der Pol and F. A. Oliehoek, “Coordinated deep reinforcement learners for traffic light control,” in Proc. NIPS Workshop on Learning, Inference and Control of Multi-Agent Systems, 2016.

[10] Moulyyy, “Traffic Signal Optimization Using Reinforcement Learning in SUMO,” GitHub repository. [Online]. Available: https://github.com/Moulyyy/Traffic_Signal_Optimization. Accessed: Oct. 28, 2026.

Note: References [1]–[9] provide background on traffic simulation and reinforcement learning. Reference [10] identifies the code repository supplied for this project. The final team should verify bibliographic details and ensure each source is cited at the appropriate point in the final text.

APPENDIX A: PROJECT FILES

File / artifact	Purpose
DQN.py	Deep Q-Network controller script
QL.py	Tabular Q-Learning controller script
Static.py	Baseline/static controller script
RL.sumocfg	SUMO scenario configuration
RL.net.xml	Network definition
RL.rou.xml	Route definitions
RL.add.xml	Additional simulation elements
det_output.xml	Detector output artifact
images/*	Graphs and diagrams supplied with repository

APPENDIX B: REPRODUCIBILITY CHECKLIST

- Install a compatible SUMO release and verify SUMO_HOME.
- Install the Python dependencies used by the scripts.
- Update hard-coded SUMO configuration paths to the local project directory.
- Run each controller with the same network, demand, simulation horizon and seed.
- Save console output, metric files, and plots for every run.
- Repeat over multiple seeds and compute mean and standard deviation.
- Document whether learning occurs during evaluation or whether a trained policy is frozen.
- Compare the final tables against raw outputs before submitting.

APPENDIX C: GUIDE / APPROVAL DETAILS TO COMPLETE

The guide name was not confirmed in the supplied messages. Replace the placeholder on the cover and certificate only after verifying the correct name and designation. The class/division,

submission date, and any department-specific signatures should also be completed before printing.

APPENDIX D: PROJECT SCHEDULE (DRAFT)

Phase	Activity	Evidence
1	Problem definition and literature survey	Topic and reference list
2	SUMO network and route configuration	XML configuration files
3	Baseline and learning-controller implementation	Python scripts
4	Simulation runs and metric collection	Console logs and output files
5	Dashboard and result visualization	Screenshots and plots
6	Testing, report writing and presentation	Final report, PPT and viva preparation